

Universidad de Concepción

Facultad de Ingeniería

Departamento de Ingeniería Eléctrica

Taller de Integración TIC

Luciano Ignacio Radrigán Figueroa

Tarea 1

Matthias Aliaga Febres

Joaquin Alonso Lozano Alamo

Concepción, 23 de octubre de 2026

Resumen

Este informe describe el sistema de adquisición remota de aceleración desarrollado para la Tarea 1 del Taller de Integración TIC (TIC3). Un ESP32-S3 lee el acelerómetro BMI270 por I^2C y envía las muestras por WiFi a una Raspberry Pi, que actúa como punto de acceso, servidor de sockets y servidor web. Desde una interfaz web en la Raspberry se configuran el rango, la frecuencia de muestreo, el modo de lectura (secuencial o paralelo), el protocolo de transporte (TCP o UDP) y la duración de cada sesión; los datos se grafican en tiempo real y se guardan en la carpeta `sensores/`.

Se explican la arquitectura, el protocolo binario diseñado y las decisiones que permiten operar a la frecuencia máxima del sensor (1600 Hz): la lectura por la FIFO del BMI270, el aumento del bus I^2C a 400 kHz y el envío por lotes (*batching*) en UDP. Con el diseño final se reciben las 1617 muestras/s que entrega realmente el sensor, sin pérdidas en TCP ni en UDP. Finalmente, se comparan ambos protocolos mediante capturas de Wireshark y se analiza el efecto del rango y la frecuencia de muestreo sobre los datos.

Índice general

Resumen I

1. Introducción y requisitos 1

2. Arquitectura y protocolo 1

3. Firmware del ESP32: decisiones y cambios 2

4. Servidor y GUI en la Raspberry Pi 3

5. Resultados 4

5.1. Rendimiento 4

5.2. Comparación TCP vs UDP con Wireshark (RF9) 4

5.3. Efecto del rango y de la frecuencia 5

6. Limitaciones y conclusiones 6

A. Anexos 7

A.1. Interfaz web 7

A.2. Capturas de Wireshark 8

A.3. Sesiones de prueba en `sensores/` 8

A.4. Uso de herramientas de IA 8

Introducción y requisitos

Se desarrolló un sistema de adquisición remota de aceleración: un ESP32-S3 lee el BMI270 por I^2C y envía las muestras por WiFi a una Raspberry Pi, que emite la red, aloja el servidor y ofrece una GUI web para configurar, graficar y guardar los datos. La Tabla 1.1 indica cómo se cumple cada requisito funcional.

Tabla 1.1: Requisitos funcionales y su implementación.

RF1	Lectura de los 3 ejes por I^2C (400 kHz) desde la FIFO del BMI270.
RF2/RF3	Modo secuencial (una tarea lee y luego envía) y paralelo (tarea lectora en el núcleo 1 + tarea emisora, unidas por una cola FreeRTOS).
RF4/RF5	La Raspberry emite la red (`10.42.0.0/24`, Pi = `10.42.0.1`) y es siempre el servidor; el ESP32 siempre inicia las conexiones.
RF6/RF7	Mensaje CONFIG con transporte (TCP por defecto), modo, frecuencia, rango y cantidad de muestras o duración, elegidos en cada sesión.
RF8	GUI web en la Raspberry (puerto 8080): configuración, gráfico en vivo y archivos CSV/JSON en `sensores/`.
RF9	Capturas de Wireshark de sesiones TCP y UDP a 100 y 1600 Hz (Sección 5.2).

Arquitectura y protocolo

La Figura 2.1 muestra la topología: dos enlaces físicos (I^2C y WiFi) y tres canales lógicos. El ESP32 inicia las conexiones porque conoce la IP fija de la Raspberry, mientras que su propia IP la asigna el DHCP.

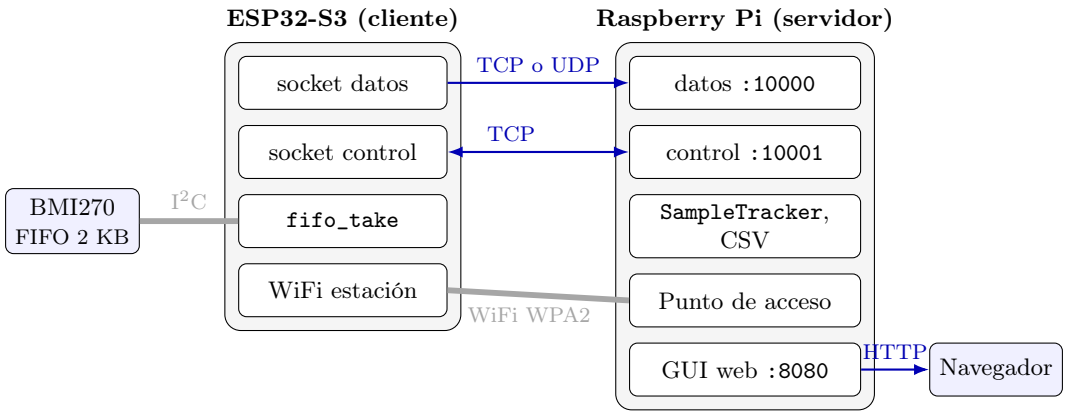


Figura 2.1: Topología. En gris, enlaces físicos; en azul, canales lógicos (comparten la misma radio).

Dos canales hacia el ESP32. Los mensajes de control son pocos y no pueden perderse; las muestras son muchas y su transporte debe ser elegible (RF6). Por eso el control va siempre por TCP, en una conexión que el ESP32 abre al arrancar y mantiene entre sesiones, y el canal de datos se abre en cada sesión con el transporte pedido. Así, elegir UDP nunca arriesga la configuración ni el cierre, y el canal de datos solo transporta muestras, sin byte de tipo.

Protocolo binario (Tabla 2.1). Mensajes de tamaño fijo, little-endian y sin relleno: estructuras `packed` con `_Static_assert` en C y prefijo `'<'` de `struct` en Python (sin él, CONFIG ocuparía 12 bytes en vez de 9). Una muestra ocupa 14 bytes frente a unos 60 en JSON; el ESP32 no formatea texto y la Raspberry decodifica con un solo `struct.unpack`. El tamaño fijo resuelve el entramado: TCP es un flujo sin límites entre mensajes, por lo que se leen exactamente 14 bytes (`recv_exact`); en UDP cada datagrama se corta en trozos de 14. Los ejes se envían crudos (`int16`, 6 bytes en vez de 12) y la Raspberry los convierte con el rango que configuró:

$$a\,[\mathrm{m/s^2}] = \mathrm{crudo} \cdot \frac{\mathrm{rango} \cdot 9{,}80665}{32768}. \tag{2.1}$$

`seq` numera cada muestra desde 0 y permite contar muestras perdidas, tardías y duplicadas, también en UDP y también si se pierden antes de la red. El `total` de END permite calcular la pérdida en el modo por tiempo, y STOP (`0xFE`) no se confunde con el primer byte de CONFIG (2/4/8/16), así que un STOP tardío se reconoce y descarta.

Tabla 2.1: Mensajes del protocolo.

Mensaje	Canal y sentido	Bytes	Contenido
CONFIG	control, Pi → ESP32	9	rango, código de frecuencia (registro), transporte, modo, tipo de fin, límite (`uint32`)
ACK	control, ESP32 → Pi	1	0 = aceptada; 1–7 = campo inválido
DATA	datos, ESP32 → Pi	14	`seq`, `t_us` (`uint32`); a_x, a_y, a_z crudos (`int16`)
STOP	control, Pi → ESP32	1	`0xFE`
END	control, ESP32 → Pi	6	`0xFF`, resultado, total de muestras tomadas

Robustez. Si el ESP32 se desconecta, el servidor no cae: vuelve a esperar una conexión y la GUI lo informa. El ESP32 reconecta el WiFi y reintenta el control cada 2 s.

Firmware del ESP32: decisiones y cambios

Del código base (driver I^2C, inicialización del BMI270, `acc_conf` y lectura de **una muestra por llamada**) solo se modificó la velocidad del bus, de 100 a 400 kHz. Lo demás es nuevo.

Límite de la lectura muestra a muestra. A 1600 Hz llega una muestra cada $T = 0{,}625$ ms y, sin FIFO, el sensor guarda solo la última. Con un costo c por vuelta, la tasa es $\min(f, 1/c)$. En la primera versión se midió $c \approx 2{,}9$ ms: $1600 \times 2{,}9$ ms $= 4{,}6$ s de trabajo por segundo, imposible. Se recibían ≈350 muestras/s y el resto se sobrescribía sin aviso. Influían el bus (9 períodos por byte; una muestra son unos 13 bytes, ≈1,2 ms a 100 kHz) y la espera con `vTaskDelay(1)`: FreeRTOS no duerme menos de un *tick* (1 ms), lo que limita a una muestra por tick, ≈1000/s, aun con un bus infinitamente rápido.

Lectura por FIFO. El BMI270 guarda cada muestra en su FIFO de 2 KB. Cada ≈5 ms, `fifo_take` lee cuántos bytes hay, los trae en una sola transferencia y separa los frames: `0x84`+6 B es una muestra; `0x40`+1 B indica muestras descartadas por FIFO llena, y `seq` avanza igual para que el servidor vea el hueco. Con un ciclo real de ≈8 ms, cada lectura trae $n \approx f\,t_{\mathrm{ciclo}} \approx 13$ muestras: 125 lecturas/s de ≈3 ms, ≈0,4 s de trabajo por segundo, con 180 ms de margen (290 muestras) si el envío se atrasa. Aun así, 100 kHz no alcanzaría (1600×7 B $\times 9 \approx 100\,800$ períodos/s); por eso el cambio a 400 kHz.

Tiempo de cada muestra. Los frames no traen hora; solo se conoce el instante de lectura t_{read}. `stamp_batch` ubica la muestra más nueva en $t_{\mathrm{read}} - \min(T, P)/2$ ($P = 5$ ms) y las anteriores un período T atrás. En lecturas siguientes, cada muestra va un paso después de la anterior, con

paso $= T + \text{error}/(8(n + \text{skip}))$ acotado a $[T/2, 2T]$, y se reancla si el error supera 20 ms. Así el tiempo nunca retrocede y sigue la frecuencia real: 618,4 µs, es decir, 1617 Hz (el oscilador del sensor es ≈1 % rápido en todas las frecuencias). Por construcción, `dt` es casi constante y no mide el *jitter* de red.

Modos. En el secuencial, una tarea lee, envía y espera 5 ms; un envío lento atrasa la lectura y la FIFO lo absorbe. En el paralelo, una tarea lectora (núcleo 1, prioridad 5) llena una cola de 512 muestras (≈320 ms) y otra tarea envía; si la cola se llena, la muestra se descarta pero conserva su `seq`. Con la FIFO ambos modos lograron 0 % de pérdida a 1600 Hz; el paralelo tolera detenciones más largas.

Envío por lotes (*batching*) en UDP. Con la FIFO, TCP no perdía muestras, pero UDP perdía entre 17 y 48 % (Figura 5.1a): se enviaba un datagrama por muestra. Cada paquete tiene un costo fijo, independiente de su tamaño: `sendto`, encabezados en lwIP, paso entre tareas hasta el driver WiFi y, en la radio, espera de canal libre, preámbulo y ACK 802.11. El ESP32 alcanzaba $\mu \approx 810$ datagramas/s (≈1,2 ms cada uno) frente a $\lambda = 1617$ pedidos; tras llenar sus buffers ($Q \approx 540$ muestras), la pérdida tiende a

$$\text{pérdida}(T) = 1 - \frac{\mu T + Q}{\lambda T} \xrightarrow{T\to\infty} 1 - \frac{\mu}{\lambda} \approx 50\ \%. \tag{3.1}$$

Ahora `send_samples` envía cada lectura de la FIFO en un solo datagrama (máximo 100 muestras = 1400 B, bajo la MTU): ≈125 datagramas/s, es decir, 125 × 1,2 ms ≈ 0,15 s de trabajo por segundo frente a 1,9 s. El límite eran los paquetes, no los bytes. TCP no tenía el problema porque agrupa los bytes por sí solo (Nagle) y, con los buffers llenos, espera en lugar de descartar. Como la estructura es `packed`, el arreglo de muestras es contiguo y se pasa directo a `sendto`; la Raspberry acepta datagramas de tamaño múltiplo de 14.

Servidor y GUI en la Raspberry Pi

Recepción. `SampleTracker` clasifica cada muestra por su `seq` (en orden, tardía, duplicada) y, con el total de END, calcula las perdidas. En UDP, `select()` atiende datos y control a la vez, se esperan 0,2 s tras END (que viaja por TCP y puede adelantarse) y solo se aceptan datagramas de la IP del ESP32.

Almacenamiento. Cada sesión genera en `sensores/` un CSV con una fila por muestra (hora de llegada, `seq`, tiempos del ESP32 y de llegada `rx_s`, aceleraciones en m/s^2 y crudas, y rango, frecuencia, modo y transporte en cada fila) y un JSON con la configuración y el resumen final.

Hilos y GUI web. Un hilo de adquisición atiende al ESP32 y escribe los archivos; el servidor HTTP crea un hilo por solicitud, de modo que la interfaz nunca retrasa la adquisición (un *lock* protege el estado compartido; un segundo *Start* recibe el código 409). Se eligió una GUI web, accesible sin instalar nada desde cualquier equipo de la red. Los comandos viajan como `POST /start` y `POST /stop`; los datos, con *Server-Sent Events*: una respuesta HTTP que no termina, por la que el servidor publica eventos JSON y que el navegador reconecta solo. Es más simple que un WebSocket, que sería bidireccional sin necesidad, y más eficiente que consultar periódicamente. `LiveFeed` agrupa las muestras y publica un lote cada 0,1 s (≈160 muestras): llegan todas, en 10 mensajes por segundo. Un `Hub` mantiene una cola por página abierta, por lo que una página lenta no afecta al resto. La configuración se valida en tres capas: formulario, Raspberry (código 400) y ESP32 (ACK 1–7). La Figura A.1 del anexo muestra la interfaz.

Resultados

5.1. Rendimiento

La Tabla 5.1 resume las tres etapas del firmware (sesiones de 10 s a 1600 Hz). Al resolver la lectura con la FIFO, el cuello de botella se trasladó al envío UDP, resuelto luego con el envío por lotes. Con el diseño final, el retardo acumulado se mantuvo bajo 10 ms (antes UDP acumulaba 0,65 s) y el *jitter* extremo a extremo, calculado con `rx_s` según RFC 3550, fue de 1,1–2,9 ms en UDP y de 1,6–3,6 ms en TCP, sin diferencia concluyente.

Tabla 5.1: Muestras recibidas por segundo y pérdida informada a 1600 Hz.

Etapa del firmware	TCP	UDP	Pérdida informada
1. Una muestra por lectura, 100 kHz	≈350/s	302–394/s	0 % (silenciosa, en el sensor)
2. FIFO + 400 kHz, un datagrama por muestra	1617/s	840–1330/s	TCP 0 %; UDP 17–48 %
3. FIFO + 400 kHz + envío por lotes	1617/s	1617/s	0 % (10 sesiones)

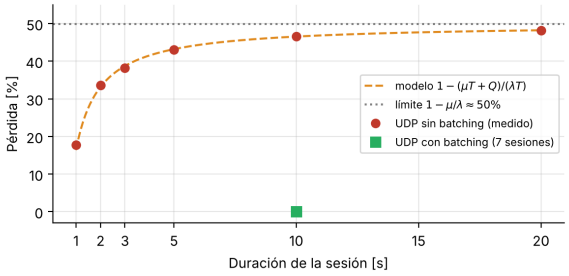
5.2. Comparación TCP vs UDP con Wireshark (RF9)

Se capturó en la Raspberry una sesión TCP y una UDP a ±2 g, 100 Hz y 1000 muestras, y a ±8 g, 1600 Hz y 10 s, en modo paralelo (Tabla 5.2; capturas en el anexo). Los mensajes de control se leen directamente: CONFIG `08 0C 00 01 01 10 27 00 00` (±8 g, 1600 Hz, TCP, paralelo, 10 000 ms), ACK `00` y END `FF 00 2D 3F 00 00` (completa, 16 173 muestras).

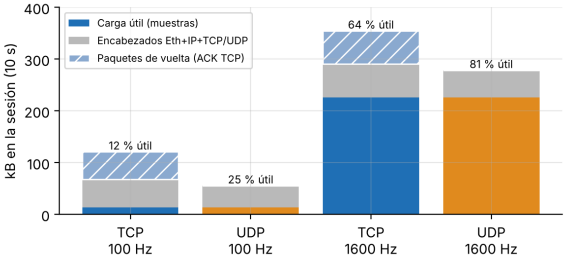
Tabla 5.2: Canal de datos en las capturas (bytes a nivel de trama en la interfaz de la Raspberry).

	100 Hz		1600 Hz	
	TCP	UDP	TCP	UDP
Paquetes con datos (ESP32 → Pi)	992	1000	1189	1251
ACK de vuelta (Pi → ESP32)	994	0	1191	0
Muestras por paquete	1	1	12–16 (máx. 40)	12–14
Conexión y cierre	SYN, FIN	—	SYN, FIN	—
Retransmisiones / duplicados	0 / 0	– / 0	0 / 0	– / 0
Muestras recibidas	1000	1000	16 173	16 173
Carga útil / bytes totales	12 %	25 %	64 %	81 %

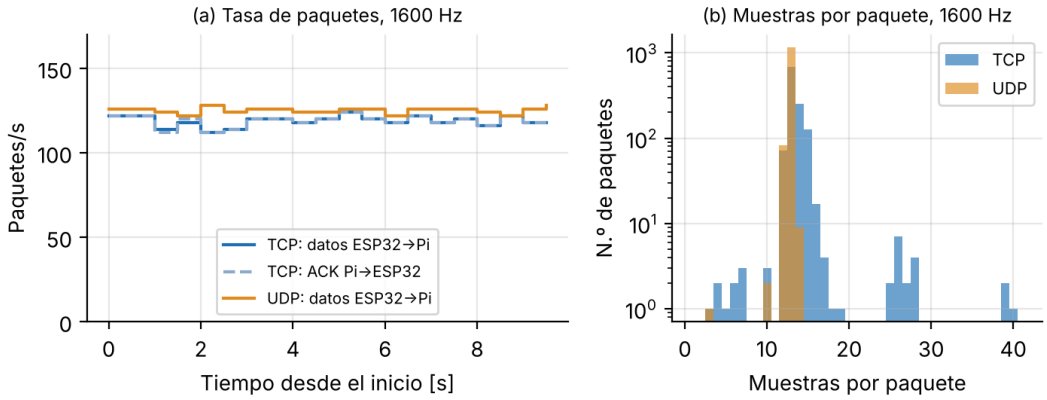
Análisis. UDP solo transmite datos; TCP agrega el saludo de tres vías, el cierre con FIN y, aquí, un ACK por segmento. Por eso transmite más bytes para la misma información, sobre todo a 100 Hz (una muestra por paquete: 12 % útil frente a 25 %). A 1600 Hz, con unas 13 muestras por paquete, la eficiencia $\eta = 14n/(14n + H)$ sube al 64 y al 81 %. Con enlace corto y baja carga (≈0,2 Mbit/s), ambos entregaron el 100 % de las muestras; con pérdidas en el enlace, TCP retransmitiría (más retardo y bloqueo del flujo) y UDP perdería muestras, que `seq` contabilizaría.



(a) Pérdida UDP a 1600 Hz antes y después del envío por lotes.



(b) Bytes por sesión en las capturas.



(c) Capturas a 1600 Hz: paquetes por segundo y muestras por paquete.

Figura 5.1: (a) El modelo de cola (discontinuo) sigue las pérdidas medidas sin parámetros ajustados. (b) Eficiencia de cada protocolo. (c) TCP envía un ACK por segmento y agrupa a su criterio (Nagle); UDP envía una lectura de la FIFO por datagrama.

5.3. Efecto del rango y de la frecuencia

Rango. El crudo de 16 bits cubre el rango elegido (ec. 2.1): la misma gravedad da la mitad de cuentas cada vez que el rango se duplica, mientras que en m/s^2 se mantiene (Tabla 5.3). Un rango pequeño da mejor resolución pero se satura antes: una sesión a ±2 g con movimiento brusco alcanzó 32 767 (0,03 % de muestras recortadas).

Frecuencia. Permite ver vibraciones hasta $f/2$ (Nyquist): 800 Hz a 1600 Hz. A cambio, aumentan los datos ($14f$ B/s: 22,6 kB/s a 1600 Hz) y el ruido, que crece como $\sqrt{f}$ porque el filtro del sensor deja pasar una banda proporcional a f. En reposo, a ±16 g, σ pasó de 4,5 a ≈9 cuentas entre 400 y 1600 Hz: un factor $2 = \sqrt{1600/400}$. Para movimientos lentos conviene un rango y una frecuencia bajos; para impactos o vibraciones, altos.

Tabla 5.3: Efecto del rango (eje Z, 1 g) y de la frecuencia (ruido en reposo), medidos en `sensores/`.

Rango	Cuentas/g teór.	Medidas	1 cuenta	Configuración	f medida	σ
±2 g	16 384	16 255	0,60 mm/s^2	±2 g, 12,5 Hz	13 Hz	0,014 m/s^2
±4 g	8 192	7 885	1,20 mm/s^2	±16 g, 400 Hz	404 Hz	0,022 m/s^2
±8 g	4 096	3 917	2,39 mm/s^2	±16 g, 1600 Hz	1617 Hz	0,043 m/s^2
±16 g	2 048	1 982	4,79 mm/s^2			

Limitaciones y conclusiones

Limitaciones. `t_us` es estimado; el frame *sensortime* del BMI270, hoy desactivado, permitiría fecharlo mejor. El valor rotulado *jitter* en la GUI es la desviación de `dt`, no el *jitter* de red. La tasa de la lectura sin FIFO a 400 kHz (≈500/s) es estimada. No se confirmó el origen de los datagramas UDP duplicados (13–26 por sesión, detectados y descartados), y las mediciones de ruido se basan en pocas sesiones en reposo.

Conclusiones. El sistema cumple RF1–RF9. Las decisiones de mayor impacto siguen un mismo principio: pagar el costo fijo de cada operación (una transacción I^2C o un paquete) una vez por grupo de muestras y no por cada muestra. La FIFO y el bus a 400 kHz elevaron la adquisición de ≈350 a 1617 muestras/s; el envío por lotes eliminó la pérdida de casi el 50 % en UDP; y el protocolo binario con `seq` y `total` permite contabilizar cualquier pérdida. Wireshark confirma las diferencias teóricas entre TCP (conexión, ACK por segmento, agrupamiento propio) y UDP (solo datagramas).

Anexos

A.1. Interfaz web



Figura A.1: Interfaz web de configuración y visualización (RF8).

A.2. Capturas de Wireshark



(a) TCP: datos y ACK.



(b) UDP: datagramas sin respuesta.



(c) Mensajes de control.



(d) Estadísticas.

Figura A.2: Capturas de Wireshark tomadas en la Raspberry Pi.

A.3. Sesiones de prueba en `sensores/`

La carpeta contiene unas 60 sesiones con rangos de ±2 a ±16 g, frecuencias de 12,5 a 1600 Hz, ambos transportes y modos, y de 127 a 32 347 muestras. Algunas representativas:

Sesión	Rango	Frecuencia	Transporte	Modo	Muestras	Pérdida
`20261008_203753`	±2 g	12,5 Hz	TCP	sec.	127	0 %
`20261008_222936`	±2 g	100 Hz	TCP	par.	506	0 %
`20261008_223001`	±16 g	400 Hz	TCP	par.	2 022	0 %
`20261008_222911`	±16 g	800 Hz	TCP	par.	4 044	0 %
`20261008_222809`	±4 g	1600 Hz	TCP	sec.	16 175	0 %
`20261010_102615`	±2 g	1600 Hz	UDP	par.	16 174	0 %

A.4. Uso de herramientas de IA

Los prompts utilizados, con su etapa, la herramienta y una nota sobre cómo se usó el resultado, se documentan en `prompts.md`, en la raíz del repositorio.